Solving Linear Systems

→ show overview of lecture first

⇒ Direct methods (i.e. non-iterative, or

$$\begin{cases} 3x_1 - 7x_2 - 2x_3 + 2x_4 = -9 \\ -3x_1 + 5x_2 + x_3 = 5 \\ 6x_1 - 4x_2 + 2x_3 - 5x_4 = 7 \\ -9x_1 + 5x_2 - 7x_3 + 6x_4 = -19 \end{cases}$$

→ $A\vec{x} = \vec{b}$ $\quad A = \begin{bmatrix} 3 & -7 & -2 & 2 \\ -3 & 5 & 1 & 0 \\ 6 & -4 & 2 & -5 \\ -9 & 5 & -7 & 6 \end{bmatrix}$ $\quad \vec{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$ $\quad \vec{b} = \begin{bmatrix} -9 \\ 5 \\ 7 \\ -19 \end{bmatrix}$

↳ Solve via Gaussian Elimination $\square \sim \boxed{\text{echelon}}$ (staircase) → then use back-sub.

augmented matrix.

$$\begin{bmatrix} 3 & -7 & -2 & 2 & | & -9 \\ -3 & 5 & 1 & 0 & | & 5 \\ 6 & -4 & 2 & -5 & | & 7. \\ -9 & 5 & -7 & 6 & | & -19 \end{bmatrix}$$

keep

r_1

$r_2 - (-1)r_1$

$r_3 - (2)r_1$

$r_4 - (-3)r_1$

$$\sim \begin{bmatrix} 3 & -7 & -2 & 2 & | & -9 \\ 0 & -2 & -1 & 2 & | & -4 \\ 0 & 10 & 6 & -5 & | & 25 \\ 0 & -16 & -11 & 12 & | & -46. \end{bmatrix}$$

keep

r_1

r_2

$r_3 - (-5)r_2$

$r_4 - (+8)r_2$

$$\sim \begin{bmatrix} 3 & -7 & -2 & 2 & | & -9 \\ 0 & -2 & -1 & 2 & | & -4 \\ 0 & 0 & 1 & 1 & | & 5 \\ 0 & 0 & -3 & -4 & | & -14. \end{bmatrix}$$

keep

r_1

r_2

r_3

$r_4 - (-3)r_3$

$$\sim \begin{bmatrix} 3 & -7 & -2 & 2 & | & -9 \\ 0 & -2 & -1 & 2 & | & -4 \\ 0 & 0 & 1 & 1 & | & 5 \\ 0 & 0 & 0 & -1 & | & 1 \end{bmatrix}$$

Back sub: $x_4 = -1$, $x_3 = 6$, $x_2 = -2$, $x = -3$.

* LU factorization 'encodes' the GE process

Define: $L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ 2 & -5 & 1 & 0 \\ -3 & 8 & -3 & 1 \end{pmatrix}$ $\quad U = \begin{bmatrix} 3 & -7 & -2 & 2 \\ 0 & -2 & -1 & 2 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & -1 \end{bmatrix}$

check: $LU = A$!

Now we can solve $Ax = b$ in two steps:

$$Ax = b$$
$$LUx = b$$
$$\underbrace{Ux}_{y} = b$$

$$y_i = b_i - \sum_{j<i} l_{ij} y_j$$

solve via forward sub.

↳ 1) ~~$Ux = y$~~ $\quad Ly = b$ (notice y = the right cd. before back sub.)

↳ 2) ~~$Ly = b$~~ $\quad Ux = y$

solve via back sub.

$$x_i = \left(y_i - \sum_{j>i} u_{ij} x_j\right) / u_{ii}$$

WHY?

LU computation (only add/sub): $n(n-1) + (n-1)(n-2) + \cdots + 3 \cdot 2 + 2 \cdot 1$ (#cols, #rows)

$$= \frac{(n^3 - n)}{3} = O(n^3)$$

Solving $Ly = b$: $(n-1) + (n-2) + \cdots + 2 + 1 = \frac{n^2 - n}{2} = O(n^2)$

solving $Ux = y$: $n + (n-1) + \cdots + 2 + 1 = \frac{n^2 + n}{2} = O(n^2)$

For large problems, LU comp is the most expensive step.

Useful when $Ax_i = b_i$, $i = 1 \cdots M$. (see inverse power method)

Pivoting: zero pivot? → swap rows → how?

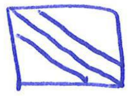
$(PA) = LU$ (LU of A with swapped rows)

* Exploit structure where possible (my PhD).

↳ SPARSITY : matrix with many zeros

form of sparsity:
- eg solving PDEs, nodal analysis, ...
- use this info in the impl of G.E. (never compute with the zeros)
- ! inverse of sparse A is usually dense

↳ Tridiagonality



$a_{ij} = 0$ for $|i-j| > 1$.

eg. solving PDEs.

! LU factorization preserves tridiag.

Compute LU of tridiag A : $O(n^2)$

solving $LUx = b$: $O(n)$.

↳ SYMMETRIC ($A = A^T$)

* EVD: $A = VDV^{-1}$ → $A = VDV^T$
* Any symm matrix : $A = LDL^T$ L is lower triangular.
* Cholesky factorization: $A = U^T U$ (upper triang.) $U = D^{1/2} \cdot L^T$.

⇒ Norms & Conditioning

< show lecture6_example2_condition

→ Use ~~[illegible]'s slides~~ Pieter's slides!

go back.

⇒ Iterative methods

Suppose $\tilde{x}$ is an approximate solution to $Ax = b$.

Then

$$A(x - \tilde{x}) \approx 0$$
$$Ax - A\tilde{x} \approx 0$$
$$b - A\tilde{x} \approx 0$$

$$A(x - \tilde{x}) = b - A\tilde{x}$$
$$x - \tilde{x} = A^{-1}(b - A\tilde{x})$$
$$x = \tilde{x} + A^{-1}(b - A\tilde{x})$$

$\tilde{x}$: approx; $A^{-1}(b - A\tilde{x})$: refinement.

? How can we avoid expensive inverses ($O(n^3)$)

↳ JACOBI method

solve $Ax = b$

write $A = D + E$ where D is "easy" to invert

In Jacobi we choose D = diag and $E = A - D$.

$$Ax = (D + E)x$$
$$Ax = Dx + Ex$$
$$Dx = Ax - Ex \quad (1)$$
$$Dx = b - Ex \quad (2)$$
$$\boxed{x = D^{-1}(b - Ex)}$$

or (2) − (1): $Dx - Dx = b - Ax$

$$Dx = Dx - (Ax - b)$$
$$\boxed{x = x - D^{-1}(Ax - b)}$$

This is the basis of an iterative method

$$x_{\text{new}}^{(n+1)} = D^{-1}(b - Ex_{\text{old}}^{(n)}) = x_{\text{old}}^{(n)} - D^{-1}(Ax_{\text{old}}^{(n)} - b)$$

Sparsity

Example For the LU-factorisation

$$LU = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0.250 & 1 & 0 & 0 & 0 \\ 0 & 0.267 & 1 & 0 & 0 \\ 0 & 0 & 0.268 & 1 & 0 \\ 0 & 0 & 0 & 0.268 & 1 \end{pmatrix} \begin{pmatrix} 4.000 & 1 & 0 & 0 & 0 \\ 0 & 3.750 & 1 & 0 & 0 \\ 0 & 0 & 3.733 & 1 & 0 \\ 0 & 0 & 0 & 3.732 & 1 \\ 0 & 0 & 0 & 0 & 3.732 \end{pmatrix}$$

we have $(LU)^{-1} = U^{-1}L^{-1}$ given by

$$\begin{pmatrix} 0.250 & -0.067 & 0.018 & -0.005 & 0.001 \\ 0 & 0.267 & -0.071 & 0.019 & -0.005 \\ 0 & 0 & 0.268 & -0.072 & 0.019 \\ 0 & 0 & 0 & 0.268 & -0.072 \\ 0 & 0 & 0 & 0 & 0.268 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ -0.250 & 1 & 0 & 0 & 0 \\ 0.067 & -0.267 & 1 & 0 & 0 \\ -0.018 & 0.071 & -0.268 & 1 & 0 \\ 0.005 & -0.019 & 0.072 & -0.268 & 1 \end{pmatrix}$$

Clearly, solving $LUx = b$ by backsubstitution is faster then computing L^{-1}, U^{-1} and $x = U^{-1}(L^{-1}b)$

Symmetric matrices

we like symm matrices, because they can always be diagonalized. (EVD always exist.)

Symmetric matrices A symmetric matrix ($A = A^T$) can be factorised $A = LDL^T$ where L is unit-lower-triangular and D is diagonal.

Positive-definite matrices A matrix is *positive definite* if $x^T Ax > 0$ whenever $x \neq 0$; equivalently, if all eigenvalues are positive, or if $A = LDL^T$ with D having strictly-positive diagonal elements.

Cholesky factorisation If A is positive definite, then there is an upper-triangular matrix U such that $A = U^T U$ (or a lower-triangular matrix L such that $A = LL^T$.)

Symmetric matrices—Example (Omit)

Example Compute the LDL^T factorisation:

$$A = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 1/2 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 2 & 1 & 0 \\ 0 & 3/2 & 1 \\ 0 & 1 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 1/2 & 1 & 0 \\ 0 & 2/3 & 1 \end{pmatrix} \begin{pmatrix} 2 & 1 & 0 \\ 0 & 3/2 & 1 \\ 0 & 0 & 4/3 \end{pmatrix}$$
$$= \begin{pmatrix} 1 & 0 & 0 \\ 1/2 & 1 & 0 \\ 0 & 2/3 & 1 \end{pmatrix} \begin{pmatrix} 2 & 0 & 0 \\ 0 & 3/2 & 0 \\ 0 & 0 & 4/3 \end{pmatrix} \begin{pmatrix} 1 & 1/2 & 0 \\ 0 & 1 & 2/3 \\ 0 & 0 & 1 \end{pmatrix} = LDL^T$$

The Cholseky factorisation is given by $A = U^T U$ where

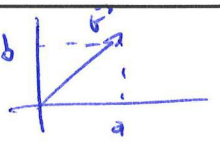
$$U = D^{1/2}L^T = \begin{pmatrix} \sqrt{2} & 0 & 0 \\ 0 & \sqrt{3/2} & 0 \\ 0 & 0 & \sqrt{4/3} \end{pmatrix} \begin{pmatrix} 1 & 1/2 & 0 \\ 0 & 1 & 2/3 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} \sqrt{2/1} & \sqrt{1/2} & 0 \\ 0 & \sqrt{3/2} & \sqrt{2/3} \\ 0 & 0 & \sqrt{4/3} \end{pmatrix}$$

→ use motivating example in Georgios' presentation.

norm = generalization of length.

Vector and matrix norms

Properties A *norm* is a measure of the *magnitude* of a vector (or matrix).



$\|\vec{v}\| = \sqrt{a^2+b^2}$

A vector norm $\|\cdot\|$ is a function $\mathbb{R}^n \to \mathbb{R}$ which satisfies

$$\|v\| \geq 0,\ \|v\| = 0 \iff v = 0;$$
$$\|\alpha v\| = |\alpha| \cdot \|v\|; \quad \|u+v\| \leq \|u\| + \|v\|.$$

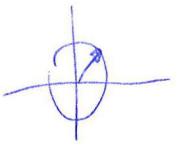
A matrix norm additionally satisfies

$$\|AB\| \leq \|A\| \cdot \|B\|.$$

Given a vector norm $\|\cdot\|_*$, the corresponding matrix norm is

fixing

$$\|A\|_* = \max\{\|Ax\|_* \mid \|x\|_* = 1\}$$



and satisfies

$$\|Ax\|_* \leq \|A\|_* \times \|x\|_*$$

Vector and matrix norms

***p*-norms** Important vector norms are

$$\|v\|_p := \left(\sum_{i=1}^n |v_i|^p\right)^{1/p}.$$
$$\|v\|_\infty := \lim_{p\to\infty} \|v\|_p.$$

Note

$$\|v\|_1 = \sum_{i=1}^n |v_i|;$$
$$\|v\|_2 = \sqrt{\sum_{i=1}^n v_i^2} = \sqrt{v \cdot v};$$
$$\|v\|_\infty = \max_{i=1,\ldots,n} |v_i|.$$

The two-norm $\|v\|_2$ gives the Euclidean length of v. The *uniform* norm $\|v\|_\infty$ gives the maximum absolute value of the components, and is usually easiest to compute.

The corresponding matrix norms are

sum of max col. elements.

$$\|A\|_1 = \sum_{i=1}^m \max_{j=1,\ldots,n} |a_{ij}|;$$
$$\|A\|_2 = \max(\mathrm{eig}(A^T A))^{1/2};$$
$$\|A\|_\infty = \max_{i=1,\ldots,m} \sum_{j=1}^n |a_{ij}|.$$

2 norm / spectral norm / largest s.v.

max of sum of col elts.

Condition number

Conditioning For a given vector norm $\|\cdot\|$ and corresponding matrix norm, define the matrix *condition number* $K(A) := ||A|| \times ||A^{-1}||$.

Suppose $\tilde{x}$ is an approximate solution to $Ax = b$. Then

$$||\tilde{x} - x|| = \|A^{-1}(A\tilde{x} - Ax)\| \leq ||A^{-1}||\ ||A\tilde{x} - b||$$

so the error satisfies:

norm of error.

$$||\tilde{x} - x|| \leq K(A) \frac{||A\tilde{x} - b||}{||A||}.$$

abs error is bounded by the cond number.

Further, since $\|A\| \cdot \|x\| \geq \|Ax\| = \|b\|$, we have $1/\|x\| \leq \|A\|/\|b\|$, so the relative error satisfies:

$$\frac{||\tilde{x} - x||}{||x||} \leq K(A) \frac{||A\tilde{x} - b||}{||b||}$$

→ cond nbr tells you something about the stiff of the problem.
if K is large, then a small change on A or b will lead to a large error on $\tilde{x}$.
input output

→ how sensitive the system is to tiny perturbations.

Iterative refinement

Approximate solution Suppose $\tilde{x}$ is an approximate solution to $Ax = b$.

Refinement Then

$$A(x - \tilde{x}) = Ax - A\tilde{x} = b - A\tilde{x} \approx 0,$$

so

$$x = \tilde{x} + A^{-1}(b - A\tilde{x}).$$

$A(x-\tilde{x}) = b - A\tilde{x}$

$x - \tilde{x} = A^{-1}(b - A\tilde{x})$

$x = \tilde{x} + A^{-1}(b - A\tilde{x})$.

Accuracy If $A^{-1}b$ can be computed less accurately than Ax, refinement typically improves the accuracy of a solution.

solving a system $O(n^3)$ → mat-vec prod. $O(n^2)$

Iterative Methods

General iterative method

Fixed-point For the linear system $Ax = b$, write $A = D + E$, where D is "easy" to invert.

Then $Ax = Dx + Ex = b$, so $Dx = b - Ex$ and

$$x = D^{-1}(b - Ex).$$

$Ax = Dx + Ex$.
$Dx = Ax - Ex$ (1)
$Dx = b - Ex$. (2)
$x = D^{-1}(b - Ex)$

Alternatively, we can write

$$x = x - D^{-1}(Ax - b).$$

$\Leftrightarrow Ax = b$

(2)−(1): $Dx - Dx = b - Ax$

$D^{-1}($ $Dx = Dx - (Ax - b)$

$x = x - D^{-1}(Ax - b)$

Update Attempt to improve x using the update

$$x' = D^{-1}(b - Ex) = x - D^{-1}(Ax - b).$$

Iteration Use this as a basis for an iterative method

$$x^{(n+1)} = D^{-1}(b - Ex^{(n)}) = x^{(n)} - D^{-1}(Ax^{(n)} - b).$$

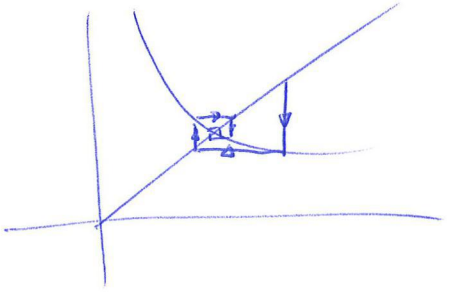
? "fixed point"

Find x such that $f(x) = 0$.

Rewrite $f(x) = 0$ to $x = g(x)$.

then use iterative scheme: $x_{i+1} = g(x_i)$

If it converges, we say that x is a fixed pt.



(Secant & NR are also fixed pt methods.)

→ easier to explain via an example.

Jacobi method

Fixed-point formula We have $\sum_{j=1}^{n} a_{ij}x_j = b_i$ for $i = 1, \ldots, n$.

Write $a_{ii}x_i + \sum_{j=1, j\neq i}^{n} a_{ij}x_j = b_i$. Rearranging gives

$$x_i = \frac{b_i - \sum_{j\neq i} a_{ij}x_j}{a_{ii}}.$$

$= \frac{1}{a_{ii}} \cdot (b_i - \sum_{j\neq i} a_{ij}x_j)$

" $D^{-1}(b - Ex)$ "

We can use this as the basis for an *iterative* method.

Jacobi Method Iterate using

$$x_i' = \frac{b_i - \sum_{j\neq i} a_{ij}x_j}{a_{ii}}.$$

An alternative formula is

$$x_i' = x_i - \frac{\sum_j a_{ij}x_j - b_i}{a_{ii}}.$$

In matrix form

$$x' = D^{-1}(b - Ex) = x - D^{-1}(Ax - b)$$

where D is the diagonal of A and $E = A - D$.

Jacobi method

Example Solve $Ax = b$ using the Jacobi method starting at $\vec{x}^{(0)} = \vec{0}$ for

$$A = \begin{pmatrix} 6 & 2 & 0 \\ 3 & 5 & -1 \\ -2 & 1 & 4 \end{pmatrix}, \quad b = \begin{pmatrix} 3 \\ 4 \\ 1 \end{pmatrix}.$$

$$x_1^{(1)} = (b_1 - a_{12}x_2^{(0)} - a_{13}x_3^{(0)})/a_{11} = (3 - 2\times 0.0 - 0\times 0.0)/6 = 0.500;$$
$$x_2^{(1)} = (b_2 - a_{21}x_1^{(0)} - a_{23}x_3^{(0)})/a_{22} = (4 - 3\times 0.0 - (-1)\times 0.0)/5 = 0.800;$$
$$x_3^{(1)} = (b_3 - a_{31}x_1^{(0)} - a_{32}x_2^{(0)})/a_{33} = (1 - (-2)\times 0.0 - 1\times 0.0)/4 = 0.250.$$

$$x_1^{(2)} = (b_1 - a_{12}x_2^{(1)} - a_{13}x_3^{(1)})/a_{11} = (3 - 2\times 0.800 - 0\times 0.250)/6 = 0.233;$$
$$x_2^{(2)} = (b_2 - a_{21}x_1^{(1)} - a_{23}x_3^{(1)})/a_{22} = (4 - 3\times 0.500 + 1\times 0.250)/5 = 0.550;$$
$$x_3^{(2)} = (b_3 - a_{31}x_1^{(1)} - a_{32}x_2^{(1)})/a_{33} = (1 + 2\times 0.500 - 1\times 0.800)/4 = 0.300.$$

Continuing yields

$$x^{(3)} = \begin{pmatrix} 0.3167 \\ 0.7200 \\ 0.2292 \end{pmatrix}, \quad x^{(4)} = \begin{pmatrix} 0.2600 \\ 0.6558 \\ 0.2283 \end{pmatrix}, \quad x^{(5)} = \begin{pmatrix} 0.2814 \\ 0.6897 \\ 0.2160 \end{pmatrix}.$$

Why would you want to do this?

Count operations: each iteration: $D^{-1}(b - Ex)$

$A: n \times n$.

Ex: mat-vec prod $O(n^2)$

$b - Ex$: $O(n)$

D^{-1}: inv: $O(n)$, multiply: $O(n)$ w. diag mat.

hence: only $O(n^2)$ ops / iteration $= O(n^2 \cdot \text{iter})$ (if A is sparse, better; if A is structured)

vs $O(n^3)$ G.E.

if iter is low then better if n is large.

Gauss-Seidel method

Gauss-Seidel Method Rather than update all the x_i simultaneously, we can update in turn. Then

$$x_i' = \frac{b_i - \sum_{j=1}^{i-1} a_{ij}x_j' - \sum_{j=i+1}^{n} a_{ij}x_j}{a_{ii}}.$$

In other words,

$$\text{for } i = 1, \ldots, n, \text{ set } x_i = \big(b_i - \textstyle\sum_{j\neq i} a_{ij}x_j\big)/a_{ii} = x_i - \big(\sum_{j=1}^{n} a_{ij}x_j - b_i\big)/a_{ii}.$$

This is more easily implemented in code:

```
for i=1:n, x(i)=x(i)-(A(i,:)*x-b(i))/A(i,i); end;
```

or using an explicit loop:

```
for i=1:n,
  ri=-b(i);
  for j=1:n, ri=ri+A(i,j)*x(j); end;
  x(i)=x(i)-ri/A(i,i);
end;
```


Gauss-Seidel method

Example Solve $Ax = b$ using the Gauss-Seidel method for:

$$A = \begin{pmatrix} 6 & 2 & 0 \\ 3 & 5 & -1 \\ -2 & 1 & 4 \end{pmatrix}, \quad b = \begin{pmatrix} 3 \\ 4 \\ 1 \end{pmatrix}, \quad x^{(0)} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

$$\begin{cases} x_1^{(1)} = (b_1 - a_{12}x_2^{(0)} - a_{13}x_3^{(0)})/a_{11} = (3 - 2\times 0.0000 - 0\times 0.0000)/6 = 0.5000; \\ x_2^{(1)} = (b_2 - a_{21}x_1^{(1)} - a_{23}x_3^{(0)})/a_{22} = (4 - 3\times 0.5000 - (-1)\times 0.0000)/5 = 0.5000; \\ x_3^{(1)} = (b_3 - a_{31}x_1^{(1)} - a_{32}x_2^{(1)})/a_{33} = (1 - (-2)\times 0.5000 - 1\times 0.5000)/4 = 0.3750. \end{cases}$$

$$\begin{cases} x_1^{(2)} = (b_1 - a_{12}x_2^{(1)} - a_{13}x_3^{(1)})/a_{11} = (3 - 2\times 0.5000 - 0\times 0.3750)/6 = 0.3333; \\ x_2^{(2)} = (b_2 - a_{21}x_1^{(2)} - a_{23}x_3^{(1)})/a_{22} = (4 - 3\times 0.3333 + 1\times 0.3750)/5 = 0.6750; \\ x_3^{(2)} = (b_3 - a_{31}x_1^{(2)} - a_{32}x_2^{(2)})/a_{33} = (1 + 2\times 0.3333 - 1\times 0.6750)/4 = 0.2479. \end{cases}$$

$$x^{(1)} = \begin{pmatrix} 0.50000 \\ 0.50000 \\ 0.37500 \end{pmatrix}, \; x^{(2)} = \begin{pmatrix} 0.33333 \\ 0.67500 \\ 0.2479 \end{pmatrix}, \; x^{(3)} = \begin{pmatrix} 0.27500 \\ 0.68458 \\ 0.21635 \end{pmatrix}, \; x^{(4)} = \begin{pmatrix} 0.27181 \\ 0.68019 \\ 0.21568 \end{pmatrix},$$

Convergence is rapid.

GS: $O(n^2 \#\text{iter})$.

Gauss-Seidel method

Example For

$$A = \begin{pmatrix} 3 & -7 & -2 & 2 \\ -3 & 5 & 1 & 0 \\ 6 & -4 & 2 & -5 \\ -9 & 5 & -5 & 6 \end{pmatrix}, \quad b = \begin{pmatrix} -9 \\ 5 \\ 7 \\ -19 \end{pmatrix}, \quad x^{(0)} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

the Gauss-Seidel method gives

$$x^{(1)} = \begin{pmatrix} -3.000 \\ 1.000 \\ 3.500 \\ -3.167 \end{pmatrix}, x^{(2)} = \begin{pmatrix} 6.778 \\ -2.500 \\ 3.083 \\ -2.417 \end{pmatrix}, x^{(3)} = \begin{pmatrix} -11.944 \\ 6.950 \\ -30.958 \\ 14.069 \end{pmatrix}, x^{(4)} = \begin{pmatrix} -4.857 \\ -6.925 \\ 119.365 \\ -66.743 \end{pmatrix}$$

so does not converge!

30 / 76

When does it work?

Gauss-Seidel method

Matrix form Write $A = L + D + U$, where L is strictly lower-triangular, D is diagonal and U is strictly upper triangular.

i.e. $L_{i,j} = 0$ if $i \leq j$, $D_{i,j} = 0$ if $i \neq j$, $U_{i,j} = 0$ if $i \geq j$.

Note that L, U here are not the L and U of the LU factorisation!!

Then the Gauss-Seidel method is given by $x' = D^{-1}(b - Lx' + Ux)$, and rearranging gives

$$\begin{aligned} x' &= (L+D)^{-1}(b - Ux) \\ &= x - (L+D)^{-1}(Ax - b). \end{aligned}$$

31 / 76

Convergence

Theorem An iterative method $x' = Tx + r$ converges if $||T|| < 1$ for some matrix norm $|| \cdot ||$.

Definition A matrix A is *diagonally-dominant* if $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ for all i.

Theorem (Convergence of Jacobi / Gauss-Seidel) If A is diagonally-dominant, then the Jacobi and Gauss-Seidel iterations converge to the solution of $Ax = b$.

Preconditioning Can *precondition* A by multiplying by a matrix P to obtain $(PA)x = Pb$.

Approximate inverse Since if $J \approx I$, then J is diagonally-dominant, precondition by $P \approx A^{-1}$ to obtain $J = PA \approx I$.

32 / 76

What to use for P? If $P \simeq A^{-1}$, then $PA \simeq I$ (hopefully diag dom).

But P should be cheap to compute.
(Sometimes possible, dep. on appl.)

why "precond" -> then P makes the cond number smaller.
if $P = A^{-1}$ then $(PA)x = Pb$. $PA = I$

κ = largest s.v. = 1 in this case.

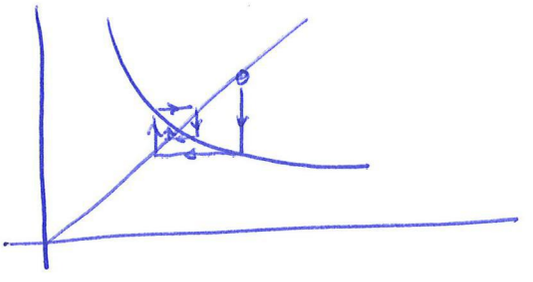
▽ This is an example of a fixed point method

Recall: Find x such that $f(x) = 0$.

Rewrite $f(x) = 0$ to $x = g(x)$

Then use $x_{i+1} = g(x_i)$

If it converges, we say that x is a fixed point.



! Secant & NR are also fixed point methods!

$$x^{(n+1)} = D^{-1}(b - Ex^{(n)})$$

↳ Element-wise, we have: $$x_i^{(n+1)} = \frac{1}{a_{ii}} \cdot \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(n)}\right).$$

$$x^{(n+1)} = x^{(n)} - D^{-1}(Ax^{(n)} - b)$$

↳ $$x_i^{(n+1)} = x_i^{(n)} - \frac{1}{a_{ii}} \left(\sum_{j} a_{ij} x_j - b_i\right)$$

Show example slide 27 / 76. (next year: Notebook?!)

Why? count operations

$$D^{-1}(b - Ex)$$

- D^{-1}: $O(n)$
- Ex: mat-vec prod $O(n^2)$
- $b - Ex$: subtraction $O(n)$

$O(n^2)$ / iteration.

$O(n^2 \cdot \text{iter})$ (Jacobi) vs $O(n^3)$ (G.E.)

If we can keep iter low, then Jacobi might be better for large n. We can also exploit structure.

↳ **Gauss-Seidel**

Already use the x_i that have been updated.

$$x_i^{(n+1)} = b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(n+1)} - \sum_{j=i+1}^{n} a_{ij} x_j^{(n)}$$

Example slide 25/26.

---

matrix form: $A = L + D + U$. (not L, U from LU fact.)

$$x^{(n+1)} = D^{-1}(b - L x^{(n+1)} + U x^{(n)})$$

* Does this always work?

No, but if A is diag dominant, then $\exists$ a GS converge.

$$\forall i: |a_{ii}| > \sum_{j \neq i} |a_{ij}|$$

* Preconditioning: (A method to speed up).

Solve $(PA)x = Pb$ instead.

In such a way that PA is 'more' diag dom.

What to choose for P?

If $P \simeq A^{-1}$, then $PA \simeq I$, but P should be cheap to compute.

* SOR (another method to speed up)

$$x_i^{(n+1)} = x_i^{(n)} + \omega \cdot \frac{b_i - \sum_{j<i} a_{ij} x_j^{(n+1)} - \sum_{j \geq i} a_{ij} x_j^{(n)}}{a_{ii}}$$

$$= (1-\omega) x_i + \omega \cdot \frac{b_i - \cdots - \cdots}{a_{ii}}$$

with $\omega \geq 1$. ($1, 1 \cdots 1, 3$).

CG -> see slides. + my notes

(make available afterwards?)

Conjugate Gradient method

= method to solve $Ax = b$ where A is pos def.

* u and v are conjugate w.r.t. A if $u^T A v = 0$.

A set of mutually conj vectors $\{v_1 \cdots v_n\}$ has the property that $v_i^T A v_j = 0$, $i \neq j$. (D)

Such a set forms a basis in $\mathbb{R}^n$

We can express the solution x^* to $Ax = b$ in this basis.

$$x^* = \sum_{i=1}^{n} \alpha_i v_i \quad \Big) \cdot A$$

$$b = Ax^* = \sum_{i=1}^{n} \alpha_i A v_i \quad \Big) \cdot v_k^T$$

$$v_k^T b = \sum_{i=1}^{n} \alpha_i v_k^T A v_i \qquad \text{use (D)!}$$

$$v_k^T b = \alpha_k (v_k^T A v_k)$$

$$\alpha_k = \frac{v_k^T b}{v_k^T A v_k}$$

⇒ Now we have an algorithm.

Find a set of mutually conj vectors.

Compute α

Find x^*.

* Crucial idea of the CG method is to carefully select the conj vectors. More precisely, we choose them in such a way that we do not need to compute all of them.

(If that's the case, then we are hopefully faster than full GE.)

* How exactly should we choose the conj vectors?

Look at the problem from an optimization POV.

$$f(\vec{x}) = \frac{1}{2} x^T A x - x^T b$$

(vector val func with unique minimizer. as. the Hessian = A and we assume A is pos def.)

$$\nabla f = Ax - b \qquad (\nabla f = 0 \Leftrightarrow Ax = b$$

hence min of f is also sol to $Ax = b$).

→ choose first basis vector in the direction of the neg. gradient. ↗ descent!

$$v_0 = b - Ax_0 = r_0 \quad \text{(residual in iter 0)}$$

→ Choose the next vectors conj to the gradient, hence, the name of the method.

→ Starting from x_0, we find the next iterate as:

$$x_{k+1} = x_k + \alpha_k p_k$$

(p_k → conj direction.)

$$p_k = r_k - \sum \beta_i p_i$$

~ G.S style "orthogonalization"

(α_k → exact Line search)

$$\nabla f(x_k + \alpha_k p_k) = 0$$

$$A(x_k + \alpha_k v_k) - b = 0.$$

$$Ax_k + \alpha_k A v_k - b \Rightarrow$$

$$\alpha_k A v_k = b - Ax_k.$$

$$\alpha_k v_k^T A v_k = v_k^T (b - Ax_k)$$

$$\alpha_k = \frac{v_k^T (b - Ax_k)}{v_k^T A v_k}.$$

* translation to actual algorithm requires some technicalities. (efficient) see course on optimization in masters.

Preconditioning

Example Let

$$A = \begin{pmatrix} 3 & -7 & -2 & 2 \\ -3 & 5 & 1 & 0 \\ 6 & -4 & 2 & -5 \\ -9 & 5 & -5 & 6 \end{pmatrix}, \quad b = \begin{pmatrix} -9 \\ 5 \\ 7 \\ -19 \end{pmatrix}, \quad P = \begin{pmatrix} 0 & 0 & -1 & -1 \\ 1 & 2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}.$$

Then

$$PA = \begin{pmatrix} 3 & -1 & 3 & -1 \\ -3 & 3 & 0 & 2 \\ 6 & -4 & 2 & -5 \\ 3 & -7 & -2 & 2 \end{pmatrix}, \quad Pb = \begin{pmatrix} 12 \\ 1 \\ 7 \\ -9 \end{pmatrix}.$$

Applying the Gauss-Seidel method to $(PA)x = (Pb)$ gives iterates:

$$x^{(1)} = \begin{pmatrix} 4.0 \\ 4.3 \\ 0.2 \\ 4.8 \end{pmatrix}, \; x^{(2)} = \begin{pmatrix} 6.9 \\ 4.0 \\ 2.9 \\ 2.1 \end{pmatrix}, \; x^{(4)} = \begin{pmatrix} 1.7 \\ 1.0 \\ 4.2 \\ 0.8 \end{pmatrix}, \; x^{(8)} = \begin{pmatrix} -1.69 \\ -1.15 \\ 5.50 \\ -0.40 \end{pmatrix}, \; x^{(12)} = \begin{pmatrix} -2.63 \\ -1.76 \\ 5.86 \\ -0.85 \end{pmatrix}.$$

Method converges slowly to $x^{(\infty)} = (-3, -2, 6, -1)^T$.

Another method to speed up the iteration

Successive Over-Relaxation Method

Successive Over-Relaxation Use slightly larger update in the Gauss-Seidel method:

$$\begin{aligned} x_i' &= x_i + \omega \frac{b_i - \sum_{j<i} a_{ij} x_j' - \sum_{j>i} a_{ij} x_j}{a_{ii}} \\ &= (1 - \omega) x_i + \omega \frac{b_i - \sum_{j<i} a_{ij} x_j' - \sum_{j>i} a_{ij} x_j}{a_{ii}}. \end{aligned}$$

with $\omega \gtrsim 1$.

Typically ω is taken to be in the range $1.1 \leq \omega \leq 1.3$.

Implement in Matlab as:

```
for i=1:n, x(i)=x(i)-omega*(A(i,:)*x-b(i))/A(i,i); end;
```

Explicitly in matrix form,

$$x' = x - \omega \, (\omega L + D)^{-1} (Ax - b).$$

In some cases it can give faster convergence rates.

Successive Over-Relaxation Method

Example Solve $Ax = b$ using the successive over-relaxation method with

$$A = \begin{pmatrix} 6 & 2 & 0 \\ 3 & 5 & -1 \\ -2 & 1 & 4 \end{pmatrix}, \quad b = \begin{pmatrix} 3 \\ 4 \\ 1 \end{pmatrix}, \quad x^{(0)} = 0, \quad \omega = 1.1.$$

First step gives $x^{(1)} = (0.5500, 0.5170, 0.4353)^T$. Second step:

$$\begin{aligned} x_1^{(2)} &= x_1^{(1)} - \omega(a_{11}x_1^{(1)} + a_{12}x_2^{(1)} + a_{13}x_3^{(1)} - b_1)/a_{11} \\ &= 0.5500 - 1.1 \times (6\times 0.5500 + 2\times 0.5170 + 0\times 0.4353 - 3)/6 = 0.3054; \\ x_2^{(2)} &= (a_{21}x_1^{(2)} + a_{22}x_2^{(1)} + a_{23}x_3^{(1)} - b_2)/a_{22} \\ &= 0.5170 - 1.1 \times (3\times 0.3054 + 5\times 0.5170 - 1\times 0.4353 - 4)/5 = 0.7225; \\ x_3^{(2)} &= (a_{31}x_1^{(2)} + a_{32}x_2^{(2)} + a_{33}x_3^{(1)} - b_3)/a_{33} \\ &= 0.4353 - 1.1 \times (-2\times 0.3054 - 1\times 0.7225 + 4\times 0.4353 - 1)/4 = 0.2008. \end{aligned}$$

Further iterates give:

$$x^{(3)} = \begin{pmatrix} 0.25455 \\ 0.68392 \\ 0.20684 \end{pmatrix}, \; x^{(4)} = \begin{pmatrix} 0.27377 \\ 0.67642 \\ 0.21888 \end{pmatrix}, \; x^{(5)} = \begin{pmatrix} 0.27460 \\ 0.67927 \\ 0.21734 \end{pmatrix}, \quad x^{(\infty)} = \begin{pmatrix} 0.27358 \\ 0.67925 \\ 0.21698 \end{pmatrix}.$$

→ here we focus on method. Derivation see opt course, Prekopa, in masters. because you need some consistent advanced topics to fully understand. the wiki page actually has a really good explanation

Conjugate-Gradient

→ find a good video!

Conjugate-Gradient Method

Method for solving $Ax = b$ where A is positive-definite.

→ hyper subspace.

Idea Construct sequence x_k minimising residual $||Ax_k - b||_2$ in $\text{span}\{b, Ab, \ldots, A^{k-1}b\}$.

Use *inner products*

$$\langle u, S, v\rangle = \sum_{i,j=1}^n u_i S_{ij} v_j \text{ and } \langle u, v\rangle = \langle u, I, v\rangle = \sum_i^n u_i v_i.$$

$u^T S v$

Algorithm

Initialise

$x_0 = 0, \;\; r_0 = b - Ax_0, \;\; v_1 = r_0;$

Iterate for $k = 1, 2, \ldots$

$s_k = \langle r_{k-1}, r_{k-1}\rangle / \langle r_{k-2}, r_{k-2}\rangle, \;\; s_1$ unused.

$v_k = r_{k-1} + s_k v_{k-1}, \qquad v_1 = r_0;$

$t_k = \langle r_{k-1}, r_{k-1}\rangle / \langle v_k, A, v_k\rangle,$

$x_k = x_{k-1} + t_k v_k,$

$r_k = r_{k-1} - A\, t_k v_k,$

Note $r_k = b - Ax_k$ and $\langle v_i, A, v_j\rangle = 0$ for $i \neq j$.

* A is pos def if $x^TAx > 0$ (whenever $x \neq 0$) $\Leftrightarrow$ A has pos eigvals.

* 2 vectors are conjugate wrt to A if $v^TAv = 0$.

* the v_k are the conjugate vectors.

Conjugate-Gradient Method

Example Solve $Ax = b$ using the conjugate-gradient method, where

$$A = \begin{pmatrix} 6 & 3 & -1 \\ 3 & 5 & 2 \\ -1 & 2 & 4 \end{pmatrix}, \quad b = \begin{pmatrix} 3 \\ 4 \\ 1 \end{pmatrix}.$$

$$x_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}, \quad r_0 = b = \begin{pmatrix} 3 \\ 4 \\ 1 \end{pmatrix};$$

$$v_1 = r_0 = \begin{pmatrix} 3 \\ 4 \\ 1 \end{pmatrix}$$

$$t_1 = \frac{\langle r_0, r_0 \rangle}{\langle v_1, A, v_1 \rangle} = \frac{26}{220} = 0.11818$$

$$x_1 = x_0 + t_1 v_1 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} + 0.11818 \times \begin{pmatrix} 3 \\ 4 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} + \begin{pmatrix} 0.35455 \\ 0.47273 \\ 0.11818 \end{pmatrix} = \begin{pmatrix} 0.35455 \\ 0.47273 \\ 0.11818 \end{pmatrix}$$

$$r_1 = r_0 - A\, t_1 v_1 = \begin{pmatrix} 3 \\ 4 \\ 1 \end{pmatrix} - \begin{pmatrix} 6 & 3 & -1 \\ 3 & 5 & 2 \\ -1 & 2 & 4 \end{pmatrix} \times \begin{pmatrix} 0.35455 \\ 0.47273 \\ 0.11818 \end{pmatrix} = \begin{pmatrix} -0.427273 \\ 0.336364 \\ -0.063636 \end{pmatrix}$$

38 / 76

Conjugate-Gradient Method

Example

$$v_1 = \begin{pmatrix} 3.00000 \\ 4.00000 \\ 1.00000 \end{pmatrix}, \quad x_1 = \begin{pmatrix} 0.35455 \\ 0.47273 \\ 0.11818 \end{pmatrix}, \quad r_1 = \begin{pmatrix} -0.427273 \\ 0.336364 \\ -0.063636 \end{pmatrix}.$$

$$s_2 = \frac{\langle r_1, r_1 \rangle}{\langle r_0, r_0 \rangle} = \frac{0.29975}{26.00000} = 0.011529$$

$$v_2 = r_1 + s_2 v_1 = \begin{pmatrix} -0.427273 \\ 0.336364 \\ -0.063636 \end{pmatrix} + 0.011529 \times \begin{pmatrix} 3.00000 \\ 4.00000 \\ 1.00000 \end{pmatrix} = \begin{pmatrix} -0.39269 \\ 0.38248 \\ -0.05211 \end{pmatrix}$$

$$t_2 = \frac{\langle r_1, r_1 \rangle}{\langle v_2, A, v_2 \rangle} = \frac{0.29975}{0.64572} = 0.46422$$

$$x_2 = x_1 + t_2 v_2 = \begin{pmatrix} 0.355 \\ 0.473 \\ 0.118 \end{pmatrix} + 0.464 \times \begin{pmatrix} -0.393 \\ 0.382 \\ -0.052 \end{pmatrix} = \begin{pmatrix} 0.355 \\ 0.473 \\ 0.118 \end{pmatrix} + \begin{pmatrix} -0.1823 \\ 0.1776 \\ -0.0242 \end{pmatrix} = \begin{pmatrix} 0.17225 \\ 0.65028 \\ 0.09399 \end{pmatrix}$$

$$r_2 = r_1 - A\, t_2 v_2 = \begin{pmatrix} -0.4273 \\ 0.3364 \\ -0.0636 \end{pmatrix} - \begin{pmatrix} 6 & 3 & -1 \\ 3 & 5 & 2 \\ -1 & 2 & 4 \end{pmatrix} \times \begin{pmatrix} -0.1823 \\ 0.1776 \\ -0.0242 \end{pmatrix} = \begin{pmatrix} 0.109625 \\ 0.043850 \\ -0.504277 \end{pmatrix}$$

39 / 76

Conjugate-Gradient Method

Example

$$v_2 = \begin{pmatrix} -0.39269 \\ 0.38248 \\ -0.05211 \end{pmatrix}, \quad x_2 = \begin{pmatrix} 0.17225 \\ 0.65028 \\ 0.09399 \end{pmatrix}, \quad r_2 = \begin{pmatrix} 0.109625 \\ 0.043850 \\ -0.504277 \end{pmatrix}.$$

$$s_3 = \frac{\langle r_2, r_2 \rangle}{\langle r_1, r_1 \rangle} = \frac{0.26824}{0.29975} = 0.89486$$

$$v_3 = r_2 + s_3 v_2 = \begin{pmatrix} -0.24177 \\ 0.38612 \\ -0.55091 \end{pmatrix}$$

$$t_3 = \frac{\langle r_2, r_2 \rangle}{\langle v_3, A, v_3 \rangle} = \frac{0.26824}{0.63278} = 0.42390$$

Solution $x = x_3 = x_2 + t_3 v_3 = \begin{pmatrix} 0.06977 \\ 0.81395 \\ -0.13953 \end{pmatrix}$

$$r_3 = r_2 - A\, t_3 v_3 = \begin{pmatrix} 0.0 \\ 0.0 \\ 0.0 \end{pmatrix} = b - Ax \text{ (residual)}$$

40 / 76

Preconditioned Conjugate-Gradient (Non-examinable)

Preconditioning Choose a *preconditioning* matrix P.

Apply the conjugate-gradient method to the equations

$$(PAP^T)y = Pb$$

and set

$$x = P^T y.$$

A typical choice is $P = (\mathrm{diag}(A))^{1/2}$.

Jacobi preconditioner

41 / 76

$Ax = b$

$\Leftrightarrow (PA)x = Pb.$

$y = Px \Leftrightarrow x = P^T y$

$(PAP^T)y = Pb.$

Approximating Eigenvalues

square!

If $A\vec{v} = \lambda\vec{v}$ with $\vec{v} \neq \vec{0}$, then λ is an eigenvalue of A with corresponding eigenvector $\vec{v}$.

- Given $\vec{v}$, how do ☒ know it's an eigvector? check if $A\vec{v} \neq \lambda\vec{v}$, so check if matrix multipl = scalar multipl. → Easy
- Given λ, how do ☒ know it's an eigenvalue? check if $Av = \lambda v$ has a nontrivial solution: $(A - \lambda I)v = 0$ via RR. (GE.)
- Given λ, how do ☒ find $\vec{v}$: see above
- HOW DO ☒ FIND λ???
  - ↳ properties of matrices: diagonal → λ = diag. entries
    triangular → λ = diag entries.
    ☒NT: $\begin{bmatrix} 1 & ? \\ 1 & ? \end{bmatrix}$? 0 and 3.
  - ↳ characteristic eq: λ is an eg value iff $(A-\lambda I)\vec{v}=0$ has a nontrivial sol.
    hence $(A-\lambda I)$ should not be invertible.
    hence $\det(A-\lambda I)=0$.
    This is a polynomial, roots = eigvals.

Butawezo: Why is factoring so important?
because of polynomials!
Why?
because eigvals!
why!
see all the apps at the start.

↳ bounds on eigenvalues: Gersgorin Circle thm.

For $i = 1 \dots n$, there exists an eigenvalue λ_i of A within the circle:

$$\{z \in \mathbb{C} : |z - a_{ii}| \leq \sum_{\substack{j=1 \\ j \neq i}}^{n} |a_{ij}|\}. \quad \text{rows}$$

Similarly, for $j = 1 \dots n$, ----

$$\{z \in \mathbb{C} : |z - a_{jj}| \leq \sum_{\substack{i=1 \\ i \neq j}}^{n} |a_{ij}|\}.$$

Example:

$$A = \begin{pmatrix} \textcircled{10} & -2 & 1 \\ 1 & \textcircled{3} & -1 \\ 0 & 1 & \textcircled{-2} \end{pmatrix}$$

Gers: $|\lambda_1 - 10| \leq |-2| + |1| = 3 \Leftrightarrow \lambda_1 \in [7, 13]$

$|\lambda_2 - 3| \leq |1| + |-1| = 2 \Leftrightarrow \lambda_2 \in [1, 5]$

$|\lambda_3 - (-2)| \leq |0| + |1| = 1 \Leftrightarrow \lambda_3 \in [-3, -1]$

(for $\mathbb{R}$ eigvals)

Gds: $|\lambda_1 - 10| \leq |1| + |0| = 1 \Leftrightarrow \lambda_1 \in [9, 11]$ — tighter!

↳ numerical methods

power method
inverse power method
(subspace iter)
OR

} 1 eigvalue.

} some eigvals

} all eigvals.

Power method

Given $A \in \mathbb{R}^{n \times n}$. Suppose $|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq |\lambda_4| \cdots \geq |\lambda_n|$.

Also, assume A is diagonalizable so that $\mathbb{R}^n$ has a basis of eigenvectors $\{\vec{v}_1, \cdots, \vec{v}_n\}$ of A.

$\forall \vec{x} \in \mathbb{R}^n$: $\vec{x}$ = lin. comb of eigvectors.

$$\vec{x} = \alpha_1 \vec{v}_1 + \alpha_2 \vec{v}_2 + \cdots + \alpha_n \vec{v}_n.$$

$$\begin{aligned} A\vec{x} &= A(\alpha_1 \vec{v}_1 + \alpha_2 \vec{v}_2 + \cdots + \alpha_n \vec{v}_n) \\ &= \alpha_1 A\vec{v}_1 + \alpha_2 A\vec{v}_2 + \cdots + \alpha_n A\vec{v}_n \\ &= \alpha_1 \lambda_1 \vec{v}_1 + \alpha_2 \lambda_2 \vec{v}_2 + \cdots + \alpha_n \lambda_n \vec{v}_n \end{aligned} \quad \Big) \; A\vec{v}_i = \lambda_i \vec{v}_i$$

$$\begin{aligned} A(A\vec{x}) = A^2\vec{x} &= A(\alpha_1 \lambda_1 \vec{v}_1 + \cdots + \alpha_n \lambda_n \vec{v}_n) \\ &= \alpha_1 \lambda_1 A\vec{v}_1 + \cdots + \alpha_n \lambda_n A\vec{v}_n \end{aligned} \quad \Big) \; A\vec{v}_i = \lambda_i \vec{v}_i$$

$$\vdots \qquad = \alpha_1 \lambda_1^2 \vec{v}_1 + \cdots + \alpha_n \lambda_n^2 \vec{v}_n.$$

$$A^k(\vec{x}) = \alpha_1 \lambda_1^k \vec{v}_1 + \cdots + \alpha_n \lambda_n^k \vec{v}_n.$$

$$= \lambda_1^k \left(\alpha_1 \vec{v}_1 + \alpha_2 \left(\frac{\lambda_2}{\lambda_1}\right)^k v_2 + \cdots + \alpha_n \left(\frac{\lambda_n}{\lambda_1}\right)^k \vec{v}_n \right).$$

Hence, $$\lim_{k \to \infty} \frac{A^k x}{\lambda_1^k} = \alpha_1 \vec{v}_1.$$

So multiplying a vector $\vec{x}$ by A, will eventually lead to the eigvector for the dom eg value.

→ let's make a method out of that.

Choose $x^{(0)}$. (how? random, or prior knowledge)

for $n = 0 \dots N$

$$y^{(n)} = A \cdot x^{(n)}$$

$$x^{(n+1)} = \frac{y^{(n)}}{\pm \|y^{(n)}\|}$$

→ $x^{(n+1)}$ will (hopefully) converge to dom. eigenvector.

end.

$$\mu^{(n)} = \frac{y^{(n)}_{i_{max}}}{x^{(n)}_{i_{max}}}$$ = eigval approx, $Ax = \lambda x \rightarrow \lambda = \frac{\|Ax\|}{\|x\|}$ " y " " x.

variations exist: $\mu^{(n)} = \frac{\|y^{(n)}\|_2}{\|x^{(n)}\|_2}$.

$$\mu^{(n)} = \frac{x^{(n)T} A x^{(n)}}{x^{(n)T} x^{(n)}} = \frac{\langle x^{(n)}, A, x^{(n)} \rangle}{\langle x^{(n)}, x^{(n)} \rangle}$$

" $\|x\|_A^2$. " $\|\cdot x\|^2$.

if $\|x^{(n)}\|_2 = 1$, then simplification.

$$\mu^{(n)} = x^{(n)T} A x^{(n)} = x^{(n)T} \cdot y^{(n)}.$$

→ example slides: 42 / 76.

Inverse Power Method

I want the smallest λ, not the biggest!

Note $Ax = \lambda x$

$A^{-1}Ax = \lambda A^{-1}x$

$x = \lambda A^{-1}x$

$\frac{1}{\lambda}x = A^{-1}x$

$$\boxed{A^{-1}\vec{x} = \frac{1}{\lambda}\vec{x}}$$ eigenvalue relation!

The smallest eigenvalue of A is the largest of A^{-1}, hence, apply power method to A^{-1} instead of A:

$$y^{(n)} = A^{-1}x^{(n)}$$! A^{-1} explicit: $O(n^3)$ = expensive!!!

$$x^{(n+1)} = \frac{y^{(n)}}{\pm\|y^{(n)}\|}$$

$$\boxed{A\,y^{(n)} = x^{(n)}}$$ solve a LS.

! ? !

Every time we solve a LS with different RHS! → LU!

$$LU\underbrace{y^{(n)}}_{z} = x^{(n)}$$

1) $Lz = x^{(n)}$
2) $Uy^{(n)} = z$.

} $O(n^2)$ / iteration instead of $O(n^3)$ / iter.

Inverse power method for any eigenvalue (not just smallest)

Suppose you have an estimate μ for some eigenvalue λ.

Then use 'spectral shift':

1) $A\vec{v} = \lambda\vec{v}$

2) $(A - \mu I)\vec{v} = A\vec{v} - \mu I\vec{v}$) $-\mu I\vec{v}$ on both sides

$(A - \mu I)\vec{v} = \lambda\vec{v} - \mu I\vec{v}$) $A\vec{v} = \lambda\vec{v}$.

$(A - \mu I)\vec{v} = (\lambda - \mu)\vec{v}$.

3) $\underline{(A-\mu I)^{-1}}\,(A-\mu I)\vec{v} = (\lambda-\mu)\underline{(A-\mu I)^{-1}}\,\vec{v}$.) same as for RPM.

$$\vec{v} = (\lambda-\mu)(A-\mu I)^{-1}\vec{v}.$$

$$(A-\mu I)^{-1}\vec{v} = \underbrace{\frac{1}{\lambda-\mu}}_{=K}\cdot\vec{v}.$$

Apply power method to $(A-\mu I)^{-1}$ to find K.

Then find λ:

K computed.

$$K = \frac{1}{\lambda - \underbrace{\mu}_{\text{given}}}$$

$$\Updownarrow$$

$$(\lambda-\mu) = \frac{1}{K}$$

$$\Updownarrow$$

$$\lambda = \mu + \frac{1}{K}$$

$$\Updownarrow$$

$$\boxed{\lambda^{(n)} = \mu + \frac{1}{K^{(n)}}}$$

Choose $x^{(0)}$

for $n = 0 \dots N$

$$y^{(n)} = (A - \mu I)^{-1} x^{(n)} \longrightarrow (A - \mu I)\, y^{(n)} = x^{(n)}$$

$$k^{(n)} = \|y^{(n)}\| \quad \text{or} \quad \frac{1}{\|x^{(n)}\|}$$

$$\left(\lambda^{(n)} = \mu + \frac{1}{k^{(n)}} \right)$$

$$x^{(n+1)} = \frac{y^{(n)}}{\pm \|y^{(n)}\|}$$

$LU\, y^{(n)} = x^{(n)}$, with z

1) $Lz = x^{(n)}$

2) $U y^{(n)} = z$

End.

! precompute: $(A - \mu I) = LU$

! Accelerate by also updating $\mu \to \mu^{(n+1)} = \mu^{(n)} + \frac{1}{k^{(n)}}$. ← cf!

→ example slides 50/76.

QR method All eigenvalues.

Definitions: $\vec{v}$ is normal if $\|v\|_2 = \sqrt{\sum_i v_i^2} = 1$ or $v^T v = v \cdot v = 1$

$\{v_i\}_{i=1}^n$ are orthogonal if $v_i \cdot v_j = 0$ for all $i \neq j$.

$\{v_i\}_{i=1}^n$ are orthonormal if orthg and normal.

Q is orthogonal if $Q^{-1} = Q^T$, i.e., the colums are orthonormal vectors. ($Q^T Q = Q^{-1} Q = I$.

(?) Given a set of vectors, how do I orthonormalize?

→ Gram-Schmidt, see my slides.

example: 55/76.

Start: $A^{(1)} = A$.

for $n = 0 \cdots N$

$Q^{(n)} R^{(n)} \leftarrow$ qr of $A^{(n)}$.

$A^{(n+1)} = R^{(n)} Q^{(n)}$

end.

This converges to an upper triangular matrix → read off λ's.

Why does it work?

$$A^{(n+1)} = R^{(n)} Q^{(n)} = \underbrace{Q^{(n)T} Q^{(n)}}_{=I} R^{(n)} Q^{(n)}$$

$$A^{(n+1)} = Q^{(n)T} \underbrace{A^{(n)}}_{=A^{(n)}} Q^{(n)}$$

so $A^{(n)}$ and $A^{(n+1)}$ are similar matrices by def., i.e, same Eigenvalues.

$$Q^{(n)} A^{(n+1)} = A^{(n)} Q^{(n)}$$

$$\tilde{T} = \underbrace{A^{(n)} Q^{(n)}}$$

multiply orthg basis with A ~ power method

Subspace Iteration (partial QR)

Use only a few vectors:

Choose $X^{(n)} \in \mathbb{R}^{n \times p}$ ← p vectors.

for $n = 0 \dots N$

$Y^{(n)} = A \cdot X^{(n)}$

$X^{(n+1)} \leftarrow$ gram-schmidt applied to $Y^{(n)}$ ← 1 vector

cf. normalization

now we need orthonormalization

→ p vectors.

end.

$\mu_i^{(n)} = \frac{\|Y_i^{(n)}\|}{\|X_i^{(n)}\|}$

Householder + Givens → see.